

Section 1. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 2. blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 3. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 4. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 5. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 6. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns.

Section 7. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. A

Section 8. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only wh

Section 9. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dict

Section 10. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost

Section 11. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adapti

Section 12. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the

Section 13. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds
Section 14. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 15. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 16. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pa
Section 17. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision En
Section 18. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a

Section 19. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 20. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing the dictionary as more files are scanned.

Section 21. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing the dictionary as more files are scanned.

Section 22. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 23. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing the dictionary as more files are scanned.

Section 24. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 25. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the input stream.

Section 26. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the input stream.

Section 27. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the input stream.

Section 28. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the input stream.

Section 29. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against the input stream.

Section 30. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern to the dictionary only if it is frequent enough to justify the overhead of maintaining it.

Section 31. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into
Section 32. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 33. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger
Section 34. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only w
Section 35. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 36. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pat

Section 37. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures

Section 38. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 39. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 40. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 41. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 42. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 43. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level fr
Section 44. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, a
Section 45. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger
Section 46. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting e
Section 47. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into
Section 48. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine prov

Section 49. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 50. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-
Section 51. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns throu
Section 52. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol
Section 53. dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost
Section 54. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual cou

Section 55. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting only patterns that pay for themselves.
Section 56. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting only patterns that pay for themselves.
Section 57. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks.
Section 58. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting only patterns that pay for themselves.
Section 59. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.
Section 60. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-

Section 61. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one c
Section 62. File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds
Section 63. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 64. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier s
Section 65. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted pattern
Section 66. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mix

Section 67. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 68. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pa
Section 69. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision
Section 70. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 71. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 72. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pay

Section 73. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine

Section 74. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-

Section 75. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, a-

Section 76. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman

Section 77. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision

Section 78. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns throu

Section 79. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern
Section 80. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level fr
Section 81. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thre
Section 82. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionar
Section 83. auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one c
Section 84. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual

Section 85. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 86. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dic
Section 87. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against a
Section 88. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine prov
Section 89. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thre
Section 90. of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision

Section 91. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding
Section 92. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural b
Section 93. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when th
Section 94. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted pattern
Section 95. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a pe
Section 96. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted pattern

Section 97. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the
Section 98. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression us
Section 99. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File C
Section 100. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 101. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 102. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the

Section 103. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 104. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 105. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte
Section 106. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier
Section 107. using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictiona
Section 108. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary agains

Section 109. it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against
Section 110. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult
Section 111. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and cod
Section 112. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 113. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa
Section 114. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte

Section 115. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 116. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 117. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 118. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 119. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself
Section 120. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 121. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 122. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 123. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 124. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 125. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 126. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 127. admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 128. a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tiered process: admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 129. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 130. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 131. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 132. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 133. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 134. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 135. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 136. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each patte
Section 137. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The eng
Section 138. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u

Section 139. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, collecting actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 140. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses a three-tier scan, collecting actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 141. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 142. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses a three-tier scan, collecting actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 143. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, collecting actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 144. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, collecting actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 145. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 146. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 147. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 148. symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 149. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 150. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree.

Section 151. larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed
Section 152. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 153. proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary
Section 154. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 155. Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine builds a p
Section 156. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent pa

Section 157. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multiple
Section 158. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine
Section 159. counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive
Section 160. frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns
Section 161. when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structures
Section 162. engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern

Section 163. scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing
Section 164. the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical
Section 165. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com
Section 166. analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns thro
Section 167. and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a thr
Section 168. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree

Section 169. Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks
Section 170. each pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patte
Section 171. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 172. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 173. growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts,
Section 174. and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File

Section 175. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 176. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 177. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 178. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 179. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 180. patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine pro

Section 181. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression

Section 182. per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the

Section 183. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using multi-

Section 184. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol

Section 185. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The en

Section 186. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E

Section 187. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d
Section 188. Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural blo
Section 189. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 190. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 191. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 192. coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Com

Section 193. a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for its
Section 194. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid
Section 195. the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression
Section 196. Adaptive File Compression using multi-level frequency analysis and a hybrid Huffman coder. The engine
Section 197. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts
Section 198. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary

Section 199. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman
Section 200. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine
Section 201. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman
Section 202. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual counts
Section 203. only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger
Section 204. hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier

Section 205. dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hy
Section 206. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itse
Section 207. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pa
Section 208. pattern only when the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns in
Section 209. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actua
Section 210. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi

Section 211. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 212. mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression using mult
Section 213. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each p
Section 214. against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffm
Section 215. into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mi
Section 216. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u

Section 217. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 218. accepted patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses the engine to compress files.

Section 219. through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 220. three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 221. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision Engine proves it pays for itself.

Section 222. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression uses the engine to compress files.

Section 223. for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual
Section 224. Decision Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing
Section 225. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 226. actual counts, and coding the resulting mixed symbol stream with one canonical hybrid Huffman tree
Section 227. Huffman coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan,
Section 228. pays for itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual

Section 229. patterns into larger structural blocks, auditing the dictionary against actual counts, and coding the res
Section 230. multi-level frequency analysis and a hybrid Huffman coder. The engine builds a per-file dictionary of f
Section 231. coder. The engine builds a per-file dictionary of frequent patterns through a three-tier scan, admitting
Section 232. the Bit Cost Decision Engine proves it pays for itself, growing accepted patterns into larger structural
Section 233. itself, growing accepted patterns into larger structural blocks, auditing the dictionary against actual co
Section 234. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E

Section 235. resulting mixed symbol stream with one canonical hybrid Huffman tree. Adaptive File Compression u
Section 236. a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only when t
Section 237. structural blocks, auditing the dictionary against actual counts, and coding the resulting mixed symbol
Section 238. frequent patterns through a three-tier scan, admitting each pattern only when the Bit Cost Decision E
Section 239. builds a per-file dictionary of frequent patterns through a three-tier scan, admitting each pattern only
Section 240. Engine proves it pays for itself, growing accepted patterns into larger structural blocks, auditing the d